Designing and Testing Quantitative Trading Algorithms in QuantConnect Using SPY

Gavin Berger
CSCI 4950
Professor Yong Wei

May 1st 2026

Senior Project Final Paper

Abstract

For my Senior Project, The focus is on designing, back testing, and evaluating quantitative trading algorithms using the Quant Connect platform and SPY, the SPDR S&P 500 ETF Trust. The purpose of the project was to explore how rule-based trading systems can be created using technical indicators, historical market data, and automated execution logic.

The project began with a short-term moving average crossover strategy, which served as the first model for testing automated trading logic. After evaluating the limitations of that approach, the project shifted toward more selective trading systems that used trend filters, pullback signals, recovery triggers, and risk management rules.

The final project compares three SPY-based algorithms to better understand how signal design, trade timing, and risk controls affect performance. Metrics such as return, drawdown, win rate, average win, average loss, Sharpe ratio, trade frequency, and expectancy were used to evaluate the strategies. The project also emphasized the importance of debugging and implementation correctness, since even a small timing issue can significantly affect backtest results.

Overall, this project demonstrates how computer science concepts such as automation, data processing, algorithm design, debugging, and performance evaluation can be applied to financial technology.

Contents

1 Introduction **5**

2 Project Goals and Motivation **6**

2.1 Project Goals 6

2.2 Motivation 6

3 Data and Research Environment **7**

3.1 QuantConnect and Lean 7

3.2 Backtest Dataset 7

3.3 Testing Setup 8

4 Algorithm 1: Creative Yellow-Green Panda **8**

4.1 Purpose of the First Strategy 8

4.2 Core Logic 8

4.3 Trading Rules 9

4.4 Strengths and Limitations 9

4.5 Algorithm 1 Results 10

5 Moving Beyond the First Strategy **12**

6 Algorithm 2: Fat Orange Chinchilla **12**

6.1 General Idea 12

6.2 Risk Management 13

6.3 Algorithm 2 Results 14

7 Algorithm 3: Alert Magenta Rat **16**

7.1 General Idea 16

7.2 Trading Rules 16

7.3 Algorithm 3 Results 17

8 Performance Metrics and Mathematical Interpretation **19**

8.1 Win Rate 19

8.2 Average Win and Average Loss 19

8.3 Drawdown 19
8.4 Sharpe Ratio 19
8.5 Expectancy 20
8.6 Profit Factor 20
8.7 Break-Even Win Rate 20

9 Comparison of Algorithms **20**

10 Debugging and Implementation Issues **22**

11 What I Learned **22**

12 Future Improvements **23**

13 Conclusion **23**

1 Introduction

This senior project explores the design and testing of quantitative trading algorithms using QuantConnect. The main asset used in the project is SPY, an exchange-traded fund that tracks the S&P 500. SPY was selected because it is liquid, widely followed, and useful as a benchmark for evaluating algorithmic trading performance.

The main goal was not simply to build a trading algorithm that produced profitable results. Instead, the goal was to understand how different trading rules affect the behavior of a strategy. A trading algorithm must be evaluated based on more than total profit. Important factors include risk, drawdown, trade frequency, win rate, average loss, and whether the strategy behaves consistently over time.

The project began with a short-term simple moving average crossover strategy. This strategy was useful because it was easy to understand and provided a foundation for learning how automated trading logic works in QuantConnect. However, after testing and reviewing the limitations of the first model, I moved toward more advanced strategies that used multiple signals and stronger risk management.

This project connects directly to computer science because it involves algorithm design, event-driven programming, data analysis, automation, debugging, and performance evaluation. Quantitative trading is a strong example of how software can be used to make decisions under uncertainty.

Figure 1: Project overview of the QuantConnect trading algorithm project.

2 Project Goals and Motivation

2.1 Project Goals

The main goals of this senior project were:

- Design and implement quantitative trading algorithms in QuantConnect.
- Use SPY historical market data to test trading strategies.
- Compare a simple baseline strategy against more advanced strategies.
- Study how technical indicators can be used to create trading signals.
- Evaluate performance using financial metrics.
- Understand how risk management affects trading outcomes.
- Identify and fix implementation issues that affect backtest reliability.

The project changed over time as I learned more about strategy design. At first, the goal was to create one working algorithm. As the project developed, it became more useful to compare multiple strategies and analyze why some approaches performed better than others.

2.2 Motivation

One of the main reasons I chose this topic was because it allowed me to apply computer science in a financial setting. Instead of building a traditional application, this project required me to work with algorithms, automation, historical data, and decision-making under uncertainty.

Financial markets are especially interesting from a software development perspective because small mistakes in logic, timing, or implementation can significantly affect the results. This made the project useful not only as a finance-related project, but also as a software engineering and debugging project.

3 Data and Research Environment

3.1 QuantConnect and Lean

The trading algorithms were developed in QuantConnect using the Lean algorithmic trading engine. QuantConnect provides a cloud-based environment for creating, testing, and evaluating automated trading strategies. It supports historical backtesting, live paper trading, technical indicators, portfolio tracking, and detailed performance reports.

One advantage of QuantConnect is that it allows historical testing and live-style evaluation within the same environment. This made it practical to move from a basic educational strategy toward more advanced algorithmic trading systems.

3.2 Backtest Dataset

The main asset used in the project was SPY. SPY was chosen because it represents broad U.S. market exposure through the S&P 500. Since it is heavily traded, it provides a useful environment for testing rule-based strategies.

Using historical trade bar data. Some strategies used daily indicators such as moving averages, RSI, EMA, and ATR. Other parts of the algorithms used minute-resolution data for execution or intraday monitoring. This allowed the project to combine longer-term market signals with more detailed trade execution.

3.3 Testing Setup

Table 1: Research and testing setup

Category	Description
Platform	QuantConnect
Engine	Lean
Programming Language	Python
Asset	SPY
Asset Type	U.S. Equity ETF
Data Type	Historical trade bars
Execution Resolution	Minute bars and consolidated bars
Indicator Resolution	15-minute and daily indicators
Starting Cash	$100,000
Main Evaluation Method	Backtesting

4 Algorithm 1: Creative Yellow-Green Panda

4.1 Purpose of the First Strategy

The first algorithm was named Creative Yellow-Green Panda. This strategy was designed as a faster SPY trading model that used short-term moving average crossovers on 15-minute price bars. The purpose of this algorithm was to test whether a medium-speed (Trade Frequency) trend-following system could react to shorter-term changes in market direction while still avoiding extremely high-frequency trading.

Instead of using daily moving averages, this strategy consolidated SPY minute data into 15-minute bars. It then calculated a 9-period simple moving average and a 21-period simple moving average. This made the strategy more responsive than a slower daily crossover model.

4.2 Core Logic

The algorithm used two simple moving averages: a 9-period SMA(ShortMovingAverage) and a 21-period SMA. The 9-period SMA represented shorter-term price movement, while the 21-period SMA represented the broader short-term trend.

When the 9-period SMA crossed above the 21-period SMA, the algorithm interpreted this as a bullish signal and entered a long position in SPY. When the 9-period SMA crossed below the 21-period SMA, the algorithm exited the position.

The strategy also included a 1% stop loss, a 2% take profit target, a 60-minute cooldown period after trades, and a maximum allocation of 95% of the portfolio. These rules helped control risk and prevent immediate re-entry after an exit.

4.3 Trading Rules

Table 2: Algorithm 1 trading rules

Rule Type	**Description**
Entry Rule	Buy SPY when the 9-period SMA crosses above the 21-period SMA
Exit Rule	Sell SPY when the 9-period SMA crosses below the 21-period SMA
Stop Loss	Exit if the position loses 1% from the entry price
Take Profit	Exit if the position gains 2% from the entry price
Cooldown	Wait 60 minutes after an exit before entering again
Maximum Allocation	95% of the portfolio
Data Resolution	15-minute consolidated SPY bars
Position Type	Long-only

4.4 Strengths and Limitations

The main strength of this strategy is that it reacts faster than a traditional daily moving average crossover. By using 15-minute bars, the algorithm can identify short-term momentum shifts and enter trades more frequently. The stop loss, take profit, and cooldown rules also make the strategy more complete than a basic crossover system.

The main limitation is that the strategy trades often. A high number of trades can increase the risk of false signals, especially when SPY is moving sideways. Short-term moving average strategies can also be sensitive to market noise, which may reduce risk-adjusted performance.

4.5 Algorithm 1 Results

Table 3: Algorithm 1 backtest results

Metric	**Result**
Starting Equity	$100,000.00
Ending Equity	$120,771.07
Net Profit	20.77%
Maximum Drawdown	9.20%
Total Orders	889
Win Rate	41%
Profit-Loss Ratio	1.72
Sharpe Ratio	-0.081
Trade Frequency	298.88 trades per year

Algorithm 1 ended with a positive return, increasing the portfolio from $100,000.00 to $120,771.07. Even though the win rate was only 41%, the strategy still produced a profit because its winning trades were larger than its losing trades on average. The profit-loss ratio of 1.72 shows that the strategy was able to offset a lower win rate with larger average gains.

However, the negative Sharpe ratio shows that the strategy's risk-adjusted performance was weak. The high number of orders also suggests that the algorithm may have been overactive. Overall, Algorithm 1 was profitable, but it showed the need for more selective trade filtering and better risk-adjusted performance.

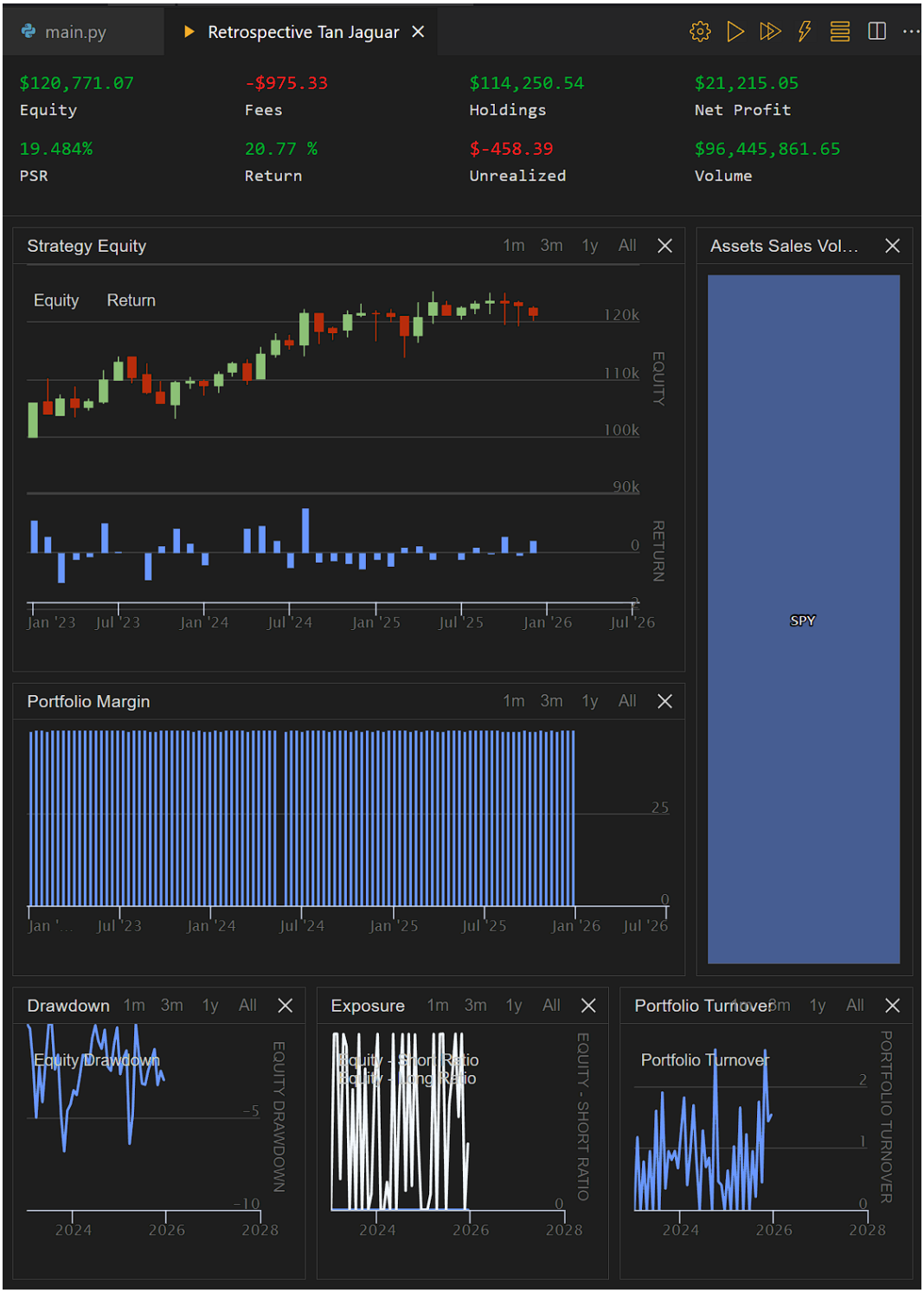

Figure 2: Algorithm 1 backtest results for the Creative Yellow-Green Panda strategy.

5 Moving Beyond the First Strategy

After testing the first moving average crossover strategy, I wanted to build strategies that reacted more selectively to market conditions. Algorithm 1 was useful because it showed how a short-term crossover system could be automated and tested. However, it also showed that frequent trading can create false signals and weaker risk-adjusted results.

The next stage of the project focused on combining multiple conditions. Instead of entering trades based only on a short-term crossover, the later strategies used broader trend filters, pullback signals, recovery triggers, and more detailed risk management rules.

This shift made the project stronger because it moved from a basic crossover model to more complete trading systems. It also allowed me to compare how different levels of trade frequency and risk control affected performance.

6 Algorithm 2: Fat Orange Chinchilla

6.1 General Idea

The second algorithm was named Fat Orange Chinchilla. This strategy was a long-only multi-factor swing trading strategy for SPY. It was designed to be more selective than Algorithm 1 by requiring multiple conditions before entering a trade.

The strategy used three main layers:

Table 4: Algorithm 2 signal structure

Layer	**Indicator**	**Purpose**
Trend Filter	50-day SMA and 200-day SMA	Confirm broader uptrend
Pullback Signal	RSI oversold condition	Identify temporary weakness
Recovery Trigger	Price above 20-day EMA	Confirm price recovery

The first condition required the 50-day SMA to be above the 200-day SMA. This helped confirm that SPY was in a broader uptrend. The second condition used RSI to identify a pullback. When RSI reached an oversold condition, the algorithm treated this as temporary weakness inside a larger uptrend. The third condition waited for price to recover above the 20-day EMA before entering a trade.

This made the strategy more selective than Algorithm 1. Instead of entering immediately on a moving average crossover, Algorithm 2 waited for a pullback inside an uptrend

and then looked for evidence of recovery.

6.2 Risk Management

Algorithm 2 included more detailed risk management rules:

Table 5: Algorithm 2 risk management rules

Risk Rule	**Purpose**
ATR-based stop loss	Adjust risk based on market volatility
Partial profit at 5% gain	Lock in part of the profit
Trailing stop after partial profit	Protect the remaining position
Maximum hold of 20 days	Keep strategy aligned with swing trading
3-day cooldown after exit	Prevent immediate re-entry

These rules made the algorithm more realistic because they controlled what happened after entering a trade. In trading systems, entry logic is only one part of the strategy. Managing the trade after entry is just as important.

6.3 Algorithm 2 Results

Table 6: Algorithm 2 backtest results

Metric	**Result**
Starting Equity	$100,000.00
Ending Equity	$104,926.22
Net Profit	4.93%
Net Profit Dollars	$4,926.22
Compounding Annual Return	1.454%
Maximum Drawdown	11.40%
Total Orders	10
Win Rate	83%
Loss Rate	17%
Average Win	2.77%
Average Loss	-8.46%
Profit-Loss Ratio	0.33
Expectancy	0.106
Sharpe Ratio	-1.084
Sortino Ratio	-0.228
Total Fees	$10.13

Algorithm 2 was more selective than Algorithm 1. It placed only 10 total orders during the backtest, which shows that the strategy waited for specific trend, pullback, and recovery conditions instead of trading frequently. The strategy finished with a positive 4.93% return, increasing the portfolio from $100,000.00 to $104,926.22.

However, the high 83% win rate did not automatically make the strategy reliable. The average winning trade was 2.77%, while the average losing trade was -8.46%. This means that one losing trade could offset several smaller winning trades. The negative Sharpe ratio and 11.40% drawdown show that the strategy still had weaknesses in risk-adjusted performance. The low trade count also makes the results harder to generalize because a small number of trades may not provide enough evidence to judge long-term reliability.

Figure 3: Algorithm 2 backtest results for the Fat Orange Chinchilla strategy.

7 Algorithm 3: Alert Magenta Rat

7.1 General Idea

The third algorithm was named Alert Magenta Rat. This strategy was developed to create a more balanced SPY trading system. It used a daily 20/50 exponential moving average crossover for trade signals and added a 200 EMA tred filter to avoid entering during weaker market conditions.

The goal was to avoid excessive trading while still capturing major upward trend movement. This strategy was slower and more selective than Algorithm 1, but it still reacted faster than a very long-term buy-and-hold approach. It also included intraday stop-loss monitoring using minute data, which allowed the algorithm to exit quickly if the position moved against it.

7.2 Trading Rules

Table 7: Algorithm 3 trading rules

Rule Type	**Description**
Entry Rule	Buy SPY when the 20 EMA crosses above the 50 EMA
Trend Filter	Enter only when price is above the 200 EMA
Exit Rule	Sell when the 20 EMA crosses below the 50 EMA
Stop Loss	Exit if the position reaches a 5% loss
Position Sizing	Uses ATR-based volatility adjustment
Signal Frequency	Daily signal check near market close
Risk Monitoring	Intraday stop-loss monitoring using minute data
Position Type	Long-only

7.3 Algorithm 3 Results

Table 8: Algorithm 3 Backtest Results

Metric	**Result**
Starting Equity	$100,000.00
Ending Equity	$131,593.01
Net Profit	31.593%
Compounding Annual Return	6.545%
Drawdown	7.60%
Total Orders	13
Win Rate	67%
Loss Rate	33%
Average Win	8.31%
Average Loss	-2.45%
Profit-Loss Ratio	3.39
Expectancy	1.927
Sharpe Ratio	0.011
Sortino Ratio	0.010
Total Fees	$13.00

These results were important because Algorithm 3 showed the strongest overall performance of the three strategies. The portfolio increased from $100,000.00 to $131,593.01, producing a 31.593% net profit. The strategy also kept drawdown at 7.60%, which was lower than Algorithm 1 and Algorithm 2.

Algorithm 3 placed only 13 orders, which shows that it was more selective than Algorithm 1. A 67% win rate combined with an average win of 8.31% and an average loss of -2.45% suggests a strong trade profile. The winning trades were much larger than the losing trades, which helped the strategy remain profitable even with a small number of trades.

The Sharpe ratio was still low, which means the strategy's risk-adjusted performance could be improved. However, compared with the other two algorithms, Algorithm 3 had the best balance of return, drawdown control, trade quality, and selectivity.

Figure 4: Algorithm 3 backtest results for the Alert Magenta Rat strategy.

8 Performance Metrics and Mathematical Interpretation

One of the most important lessons from this project is that total profit does not tell the whole story. A strategy can make money but still have poor risk-adjusted performance. For that reason, I evaluated the algorithms using several different metrics.

8.1 Win Rate

Win rate measures the percentage of trades that were profitable. A high win rate can be useful, but it does not guarantee that a strategy is strong. If the losing trades are much larger than the winning trades, the strategy can still lose money.

8.2 Average Win and Average Loss

Average win and average loss help explain trade quality. A strategy is usually stronger when the average winning trade is larger than the average losing trade.

This was one of the major differences between Algorithm 2 and Algorithm 3. Algorithm 2 had the highest win rate at 83%, but its average loss was -8.46%, while its average win was only 2.77%. This means that one losing trade could offset several winning trades. Algorithm 3 had a lower win rate of 67%, but its average win was 8.31% and its average loss was only -2.45%. This made Algorithm 3 stronger from a trade-quality perspective.

8.3 Drawdown

Drawdown measures how much the portfolio declined from a previous high. This is important because a profitable strategy may still be risky if it experiences large losses along the way.

8.4 Sharpe Ratio

Sharpe ratio measures risk-adjusted return. A higher Sharpe ratio usually means a strategy is producing better returns relative to its volatility. In this project, Sharpe ratio helped show that even profitable strategies still had room for improvement.

8.5 Expectancy

Expectancy estimates the average expected result per trade. The formula is:

$$\text{Expectancy} = (\text{Win Rate} \times \text{Average Win}) - (\text{Loss Rate} \times \text{Average Loss}) \tag{1}$$

This metric is useful because it combines win rate and trade size into one number. A strategy with a high win rate can still have weak expectancy if the losses are too large.

8.6 Profit Factor

Profit factor compares total gross profit to total gross loss.

$$\text{Profit Factor} = \frac{\text{Gross Profit}}{\text{Gross Loss}} \tag{2}$$

A profit factor above 1.0 means the strategy made more from winning trades than it lost from losing trades.

8.7 Break-Even Win Rate

Break-even win rate estimates the win rate required for a strategy to avoid losing money based on the size of the average win and average loss.

$$\text{Break-Even Win Rate} = \frac{\text{Average Loss}}{\text{Average Win} + \text{Average Loss}} \tag{3}$$

9 Comparison of Algorithms

The table below compares the three major strategies from the project.

Table 9: Comparison of project algorithms

Metric	**Algorithm 1**	**Algorithm 2**	**Algorithm 3**
Strategy Name	Creative Yellow-Green Panda	Fat Orange Chinchilla	Alert Magenta Rat
Strategy Type	15-minute 9/21 SMA crossover	Daily trend, pullback, recovery	Daily 20/50 EMA with 200 EMA filter
Main Asset	SPY	SPY	SPY
Trade Style	Medium-speed long-only	Long-only swing trading	Long-only trend-following
Trade Count	889 orders	10 orders	13 orders
Win Rate	41%	83%	67%
Average Win	1.23%	2.77%	8.31%
Average Loss	-7.32%	-8.46%	-2.45%
Max Drawdown	9.20%	11.40%	7.60%
Net Profit	20.77%	4.93%	31.593%
Sharpe Ratio	-0.081	-1.084	0.011
Profit-Loss Ratio	1.72	0.33	3.39

Based on the final results, Algorithm 3 was the strongest overall strategy. It produced the highest net profit, the lowest drawdown, and the strongest profit-loss ratio. Algorithm 3 also placed only 13 orders, which means it was selective without being inactive.

Algorithm 1 was also profitable, but it placed 889 orders. This suggests that the strategy may have been too active and more exposed to short-term market noise. Its negative Sharpe ratio also shows that the return was not strong when adjusted for risk.

Algorithm 2 had the highest win rate at 83%, but it had the lowest net profit and the highest drawdown. Its average loss was also much larger than its average win. This shows why win rate alone is not enough to judge a trading strategy.

Overall, Algorithm 3 provided the best balance between profitability, trade quality, and risk control.

10 Debugging and Implementation Issues

One of the most important parts of this project was learning that a trading strategy is only reliable if the code is implemented correctly. During the project, I found that one version of the strategy appeared to be trading only on the first trading day of each month. This suggested that there may have been a scheduling or timing issue in the algorithm.

This bug was important because it could make the backtest results misleading. If the strategy only checks for entries once per month, then many valid trading opportunities may be missed. This would affect trade count, entry timing, return, drawdown, and all other performance metrics.

This issue showed that quantitative trading is not only about financial theory. It is also a software engineering problem. A strategy can have good logic, but if the implementation is wrong, the results cannot be trusted.

Debugging this issue helped me understand the connection between code correctness and financial interpretation. Before making conclusions about an algorithm, the developer must confirm that the strategy is evaluating signals at the intended time and executing trades according to the correct rules.

11 What I Learned

QuantConnect and quantitative algorithms helped me better understand how computer science applies to financial technology. I learned how to design algorithmic trading strategies, work with historical market data, evaluate backtest results, and interpret financial metrics.

One of the biggest lessons was that algorithmic trading requires both technical skill and careful analysis. Writing the code is only one part of the process. The strategy also needs to be tested, debugged, compared, and evaluated using multiple metrics.

I also learned that a strategy should not be judged only by whether it made money in a backtest. It is important to ask deeper questions:

- Did the strategy take too much risk?
- Was the drawdown reasonable?
- Was the win rate supported by strong average wins?
- Were the losses controlled?

- Did the algorithm trade when it was supposed to?
- Were the results based on enough trades to be meaningful?

This project also improved my understanding of Python, QuantConnect, event-driven programming, and data-driven decision-making. It showed me how software can be used to automate decisions in a complex environment.

12 Future Improvements

There are several ways this project could be improved in the future.

First, I would continue improving the risk management system. This could include better stop-loss placement, volatility-based position sizing, and stronger rules for reducing exposure during unstable market conditions.

Second, I would test the strategies across different market environments. A strategy should be tested during bull markets, bear markets, and sideways markets. This would help determine whether the strategy works generally or only during specific conditions.

Third, I would compare each algorithm more directly against buy-and-hold SPY. Since SPY itself tends to increase over long periods, it is important to know whether the algorithm actually improves performance compared with simply holding the asset.

Fourth, I would continue testing the strategies in live paper trading. Paper trading would allow the algorithm to be tested in real-time market conditions without risking actual money.

Finally, I would improve the final strategy by collecting more trades. A strategy with only a small number of trades is difficult to judge. More trade data would make the results more reliable.

13 Conclusion

Overall, this research project focused on designing and testing quantitative trading algorithms using QuantConnect and SPY. The project began with a shorter-term moving average crossover strategy and later developed into more selective strategies using trend filters, pullback signals, recovery triggers, and risk management rules.

The first algorithm, Creative Yellow-Green Panda, used a 15-minute 9/21 SMA crossover strategy. It produced a 20.77% net profit, but it also placed 889 orders and had a

negative Sharpe ratio. This showed that the strategy was profitable but may have been too active.

The second algorithm, Fat Orange Chinchilla, used a daily swing-trading approach based on a broader trend filter, RSI pullback condition, and EMA recovery trigger. It produced a 4.93% net profit and had the highest win rate at 83%. However, its average loss was much larger than its average win, and its drawdown was the highest of the three algorithms.

The third algorithm, Alert Magenta Rat, showed the strongest results overall. It used a daily 20/50 EMA crossover with a 200 EMA trend filter, ATR-based position sizing, and stop-loss protection. It produced a 31.593% net profit, a 67% win rate, and a 7.60% drawdown. It also had the strongest profit-loss ratio at 3.39.

The project showed that successful algorithmic trading requires more than finding profitable trades. A strong strategy must also manage risk, control losses, avoid misleading backtest results, and be implemented correctly. The comparison between the three algorithms showed that the best-performing strategy was not the one with the highest win rate or the most trades. Instead, the strongest strategy was the one that balanced return, drawdown, trade quality, and risk management.

Overall, this project allowed me to apply computer science concepts to a real-world financial problem. It required programming, data analysis, automation, debugging, and critical thinking. The biggest takeaway is that quantitative trading is not just a finance topic; it is also a software engineering challenge where logic, implementation, and evaluation all matter.

AI Use Statement

Artificial intelligence tools were used in the writing process to assist with grammar, sentence structure, organization, and overall clarity. The original ideas, project concept, research, analysis, and conclusions presented in this report are my own. AI was used only as a writing support tool and was not used as a replacement for my original work or thinking.

References

[1] QuantConnect. *Lean Algorithmic Trading Engine Documentation*. QuantConnect. Available at: https://www.quantconnect.com/docs. Accessed April 2026.

[2] State Street Global Advisors. *SPDR S&P 500 ETF Trust*. Available at: https://www.ssga.com/us/en/intermediary/etfs/spdr-sp-500-etf-trust-spy. Accessed April 2026.

[3] OpenAI. *ChatGPT* [Large language model]. OpenAI. Available at: https://chat.openai.com/. Accessed May 2026.